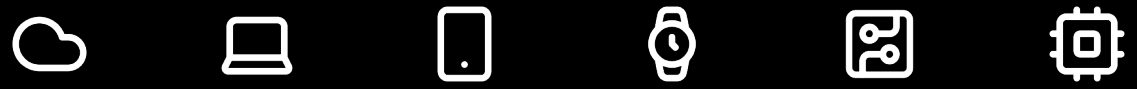


AI in Flutter

Lecture 9: on-device ML and connecting to LLMs

Dinu-Ștefan Rusu

ORIENTATION

Who's teaching this course

Dinu-Ștefan Rusu

dinu_stefan.rusu@upb.ro
Microsoft Teams course channel
rusudinu.com

45+

Flutter apps shipped

400k

installs across them

Appeared in, spoken at & partnered with

TODAY

What you should leave with

On-device or cloud	The variables that decide it: latency, privacy, offline, cost per call, capability ceiling
ML Kit and LiteRT	Real Dart for text and barcodes, plus your own quantized model through tflite_flutter
Where the API key lives	Why a key inside an app binary is public, and the two architectures that fix it
LLMs in a real UI	Streaming, timeouts, token cost, and validating output you cannot trust

PART 1 · THE DECISION

On the device, or on someone else's GPU

Five variables: latency, privacy, offline, cost, capability ceiling

THE DECISION

Three different things called “AI in the app”

Classic on-device ML

A fixed-purpose model runs locally: read text, find a barcode, locate a face. Milliseconds, offline, nothing leaves the phone.

On-device generative

A small language model on the phone's NPU: summarize, proofread, rewrite. Gated on hardware, so never guaranteed.

A cloud LLM

A large model on someone else's machines. Highest capability, network-bound, and billed per token. It is also where this lecture's security problems come from.

They are not three sizes of the same thing. They differ in where the compute happens, who pays for it, what happens with no network, and what an attacker can reach.

Picking one is an architecture decision. It is not a library choice, and it is very expensive to reverse late.

Most shipping apps use two of the three. The boundary between them is where most of the design work sits.

THE DECISION

On-device vs cloud, variable by variable

	On-device	Cloud
Latency	**1–50 ms, no network involved**	300 ms – several seconds, plus the round trip
No connection	**works, because offline is a normal state**	the feature does not exist
Where the data goes	**nowhere; it never leaves the device**	to a third party: a residency and GDPR question
Marginal cost per call	**zero, forever**	per token, forever
Capability ceiling	**whatever fits in RAM**	the largest model in production
Battery	**your CPU / GPU / NPU burns it**	the radio burns it, and radios are expensive
Updating the model	**an app update, or a download you own**	instant, server-side, no client involved

On-device first, cloud when it cannot cope. Privacy, offline and cost all point one way; only capability points the other.

THE DECISION

Callback to Lecture 1: will the model fit?

28 GB	14 GB	3.5 GB	~1 GB
7B at float32	**7B at float16**	**7B at int4**	**2B at int4**
one 4-byte float per parameter	half precision, the usual baseline	aggressive, with a real quality cost	the class that ships inside a phone

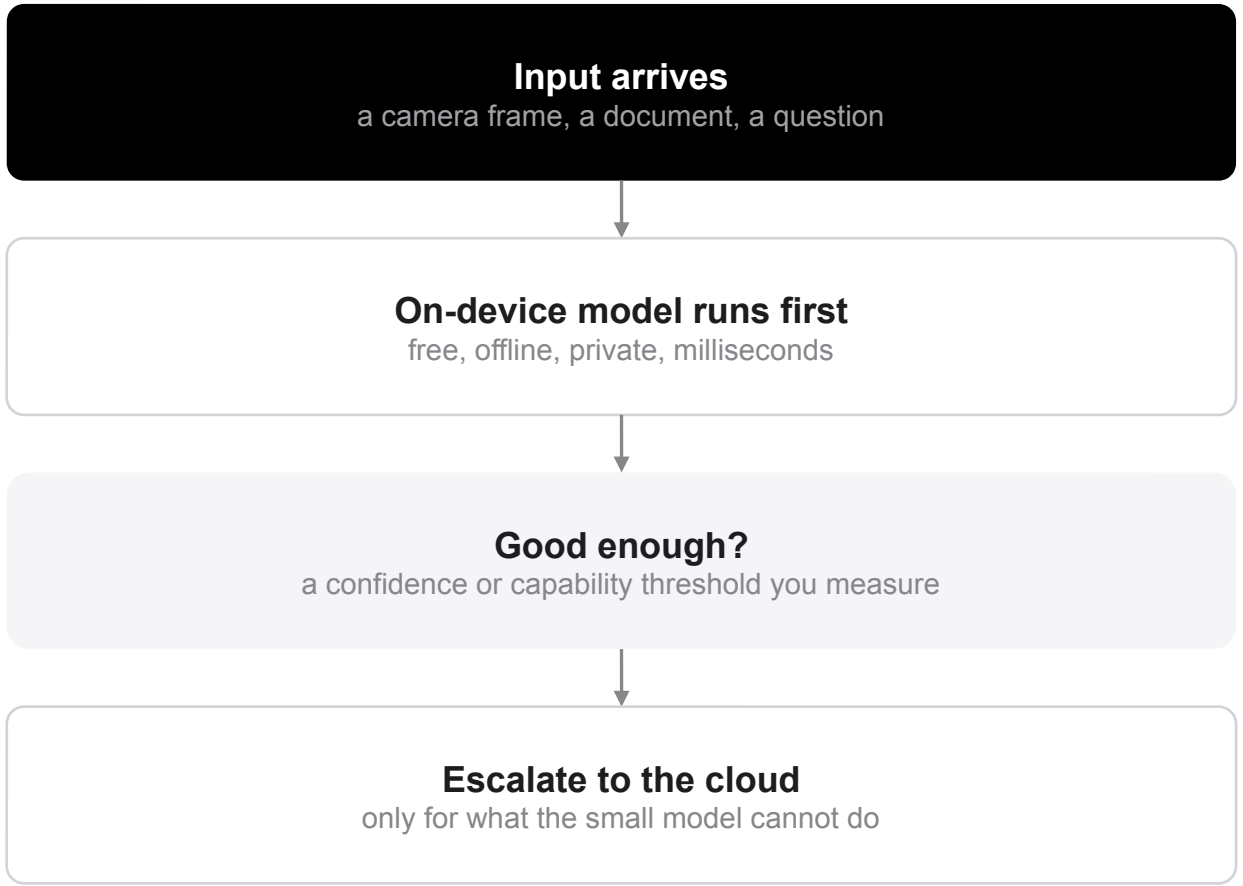
Weights come first: parameters × bytes-per-parameter is memory you must find before a single token is produced. The KV cache then grows on top of that, with every token of context you keep.

A 2026 flagship has 8–16 GB of RAM in total, shared with the OS and every other app the user expects to still be running. Android will kill your process long before you can claim half of it.

This is Lecture 1's VRAM → RAM → fail flowchart with real numbers in it. On-device generative AI is a resource-allocation problem, exactly as promised.

THE DECISION

The pattern that ships: hybrid



Scan locally, look up remotely.

ML Kit reads the barcode on the device; only the resulting 13 digits go to your API. The image never leaves.

Detect locally, understand remotely.

A face or object detector finds the region; you upload a small crop instead of a 12-megapixel frame.

Draft locally, refine remotely.

The on-device model writes the first version instantly; the cloud model is only called when the user asks for better.

Cache, then do not call at all.

The same barcode, the same photo, the same question. A local cache turns the second occurrence into zero latency and zero cost.

Know your escalation rate.
The share of requests that reach the cloud is your bill, your p95 latency and your privacy exposure, in one number. A hybrid system whose escalation rate you cannot quote is a cloud system with a cache.

PART 2 · ON THE DEVICE

ML Kit, LiteRT, and making a model fit

The problems Google already solved, and then your own

Google ML Kit: the solved problems

Text recognition

google_mlkit_text_recognition: Latin, Chinese, Devanagari, Japanese and Korean script packs. Blocks → lines → elements, each with a box.

Barcode scanning

google_mlkit_barcode_scanning: every common 1D and 2D symbology, plus the parsed payload for URLs, Wi-Fi and contacts.

Face detection

google_mlkit_face_detection: bounding boxes, landmarks, contours, smile and eye-open probabilities. Detection, never recognition.

Language ID & translation

google_mlkit_language_id and google_mlkit_translation: per-language-pair models the device downloads once and keeps.

Labeling & object detection

google_mlkit_image_labeling and google_mlkit_object_detection: coarse classification, and tracking across frames.

How you add it

One plugin per feature. The google_ml_kit umbrella drags every model into your binary. Android and iOS only. There is no web support.

Reading text from an image, end to end

```
// pubspec.yaml:  google_mlkit_text_recognition: ^0.15.0
import 'package:google_mlkit_text_recognition/google_mlkit_text_recognition.dart';

class ReceiptScanner {
  // One recognizer, reused. One per frame leaks native memory.
  final _recognizer = TextRecognizer(script: TextRecognitionScript.latin);

  Future<String> read(String filePath) async {
    final input = InputImage.fromFilePath(filePath);
    final RecognizedText result = await _recognizer.processImage(input);
    for (final block in result.blocks) {
      for (final line in block.lines) {
        debugPrint('${line.text}  @ ${line.boundingBox}');
      }
    }
    return result.text;
  }

  void dispose() => _recognizer.close();  // free the native detector
}
```

processImage is async and does its work off the Dart thread, so the UI keeps rendering while it runs.

The result is a tree: blocks of lines of elements, every node carrying a bounding box you can draw over a preview.

close() is not optional. The detector owns a native model; skipping it is a leak the Dart GC cannot see.

The first call is the slow one: the model is loaded, and for some APIs downloaded, on first use.

No network, no key, no per-call cost.

Barcodes from a live camera stream

```
final _scanner = BarcodeScanner(
    formats: [BarcodeFormat.ean13, BarcodeFormat.qrCode]);
bool _busy = false;

Future<void> _onFrame(CameraImage frame) async {
  if (_busy) return;      // drop frames, never queue them
  _busy = true;
  try {
    final input = _toInputImage(frame);   // bytes + rotation
    final codes = await _scanner.processImage(input);
    if (codes.isNotEmpty) _onCode(codes.first.rawValue);
  } finally {
    _busy = false;
  }
}
```

A camera hands you 30 frames a second. Inference does not always finish in 33 ms, so a queue grows without bound and the preview drifts behind reality.

Dropping frames is correct: the next one is nearly identical and arrives in a moment.

InputImage needs the raw planes plus the sensor rotation. Get the rotation wrong and detection silently returns nothing at all, with no error and no result.

Stop the stream and close() the scanner in dispose(), and again when the app backgrounds. A live camera together with continuous inference drains the battery quickly (→ Lecture 11).

Real-time on-device ML is a back-pressure problem. The model is fast enough; the frame pipeline around it is what breaks.

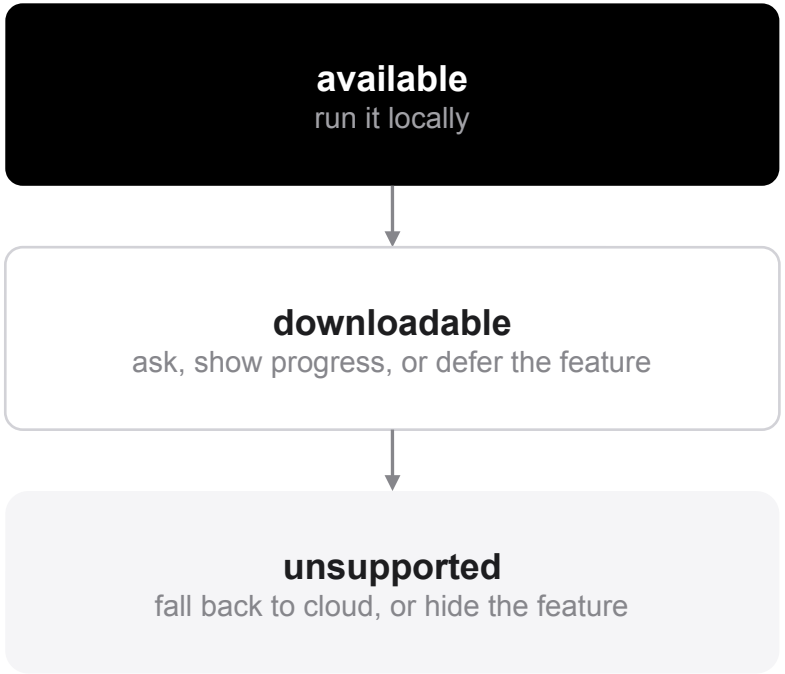
On-device generative AI is a capability, not a device

The step past classification: a small language model running on the phone's own accelerator: summarize a thread, proofread a message, rewrite a paragraph, describe a photo.

Google exposes this through the ML Kit GenAI APIs backed by Gemini Nano; Apple exposes an equivalent on-device model to apps on supported hardware. Both are the same shape from your side.

It is gated on the hardware and the OS, not on your app: enough accelerator, enough RAM, and the model itself present. The OS downloads that model and shares it between apps rather than shipping it in your binary.

Ask the SDK: three answers.



Feature-detect, never device-detect. A hard-coded list of phone models goes out of date with every hardware release. Availability is a runtime question, and the fallback path is part of the feature.

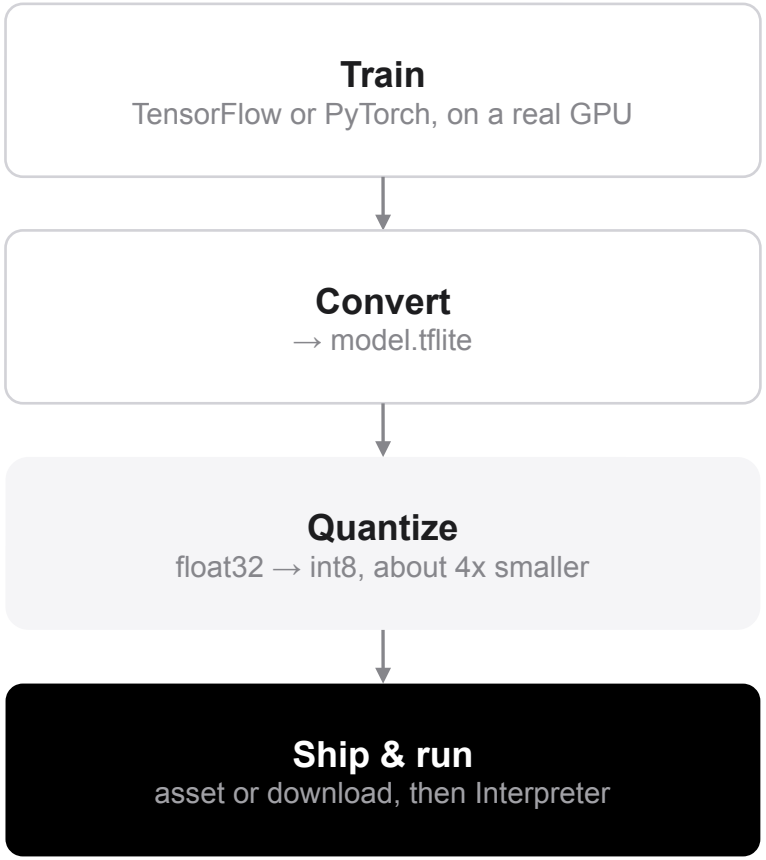
Your own model: TensorFlow Lite, now LiteRT

ML Kit covers the problems Google already solved. When the task is yours, such as grading a weld, classifying a leaf, or scoring a window of accelerometer data, you bring your own model.

LiteRT, the runtime published for years as TensorFlow Lite, executes a .tflite flatbuffer on the device. The file is the graph plus the weights: no training code, no ability to learn, fixed input and output shapes.

In Flutter, tflite_flutter binds the LiteRT C API through dart:ffi, which is a direct call into native code rather than a platform-channel message per inference (→ Lecture 12).

Delegates hand layers to the accelerator: GPU, NNAPI on Android, Core ML on iOS. Every delegate is allowed to refuse, and CPU is always the fallback, so you measure on the cheapest phone you support, not the newest.



None of the first three steps happen in Dart.

tflite_flutter: load, run, get off the UI isolate

```dart
// tflite_flutter: ^0.11.0. The .tflite ships as a Flutter asset
import 'package:tflite_flutter/tflite_flutter.dart';

late final Interpreter _interp;
late final IsolateInterpreter _iso;

Future<void> load() async {
  final o = InterpreterOptions()..threads = 2;
  try { o.addDelegate(GpuDelegateV2()); } catch (_) {}   // CPU is fine
  _interp = await Interpreter.fromAsset(
      'assets/models/leaf_int8.tflite', options: o);
  _iso = await IsolateInterpreter.create(address: _interp.address);
}

Future<List<double>> classify(List input) async {
  final out = List.filled(5, 0.0).reshape([1, 5]);
  await _iso.run([input], out);      // off the UI isolate
  return out[0].cast<double>();
}
void dispose() { _iso.close(); _interp.close(); }
```

Interpreter.run is synchronous and CPU-bound. On the UI isolate it drops frames, so hand the interpreter's native address to an isolate (→ Lecture 3).

Shapes are baked into the file at conversion time. getInputTensors() tells you what the model wants; guessing produces a crash or silent nonsense.

An int8 model expects int8 input, with the scale and zero-point traveling alongside the tensor.

close() both. Native memory again, invisible to the Dart GC.

Quantization, and how the model reaches the phone

	float32	float16	int8
Size	1x, the baseline	0.5x	**0.25x**
Accuracy	reference	loss is usually negligible	**small and task-dependent, so measure it**
CPU speed	baseline	about baseline	**2–4x faster on integer units**

Bundled as an asset.

In the store binary, available on first launch, and re-downloaded by every user every time you change it. Install size is a conversion metric, so a 90 MB model is a product decision, not a detail.

Downloaded on first run.

A smaller install, but now you own a download, a cache, a version check, a retry, and a first-launch state where the feature does not work yet. Firebase ML custom model hosting does this for you, with rollout and rollback.

Accelerator paths are int8-first. Quantization is often the difference between running on the NPU and running on the CPU.

PART 3 · CLOUD LLMS

Calling a model you do not host

Where the key lives determines the architecture; everything else is UI

The prototyping trap: a key inside the app

Do not ship this

```
// google_generative_ai is deprecated, and the wrong shape anyway.
import 'package:google_generative_ai/google_generative_ai.dart';

const apiKey = 'AIzaSyD-9tSrke72_EXAMPLE_KEY_xxxxxxxxxx';

final model = GenerativeModel(
  model: 'gemini-2.5-flash',
  apiKey: apiKey,        // <- compiled into the APK / IPA
);

final res = await model.generateContent([Content.text(prompt)]);
```

This is fine for one afternoon on an emulator. It stops being fine once the build leaves your machine.

Three separate things are wrong

The package.

google_generative_ai is deprecated; firebase_ai replaces it, and firebase_ai is also the fix for the other two problems.

The key.

Anything compiled into your app ships to every person who installs it. --dart-define, a .env file, base64, string obfuscation: all the same outcome.

The architecture.

The client is talking straight to a metered API, so nothing but the client limits who can spend against your account.

A key in a binary is a published key

```
# Anyone can run this against any app, on their own phone.
$ adb shell pm path com.example.app
package:/data/app/~~a1b2.../base.apk

$ adb pull /data/app/~~a1b2.../base.apk .
$ unzip -o base.apk -d out/

$ strings out/lib/arm64-v8a/libapp.so \
      | grep -E 'AIza[0-9A-Za-z_-]{35}'
AIzaSyD-9tSrke72_EXAMPLE_KEY_xxxxxxxxxx

# Elapsed: about a minute.
# iOS is the same exercise on the .ipa payload.
```

Release Dart is AOT-compiled into libapp.so, but string constants survive compilation. They have to: the program reads them at runtime.

Obfuscation renames symbols. It does not remove the bytes your HTTP client eventually puts on the wire, and a proxy on the attacker's own device reads the request regardless.

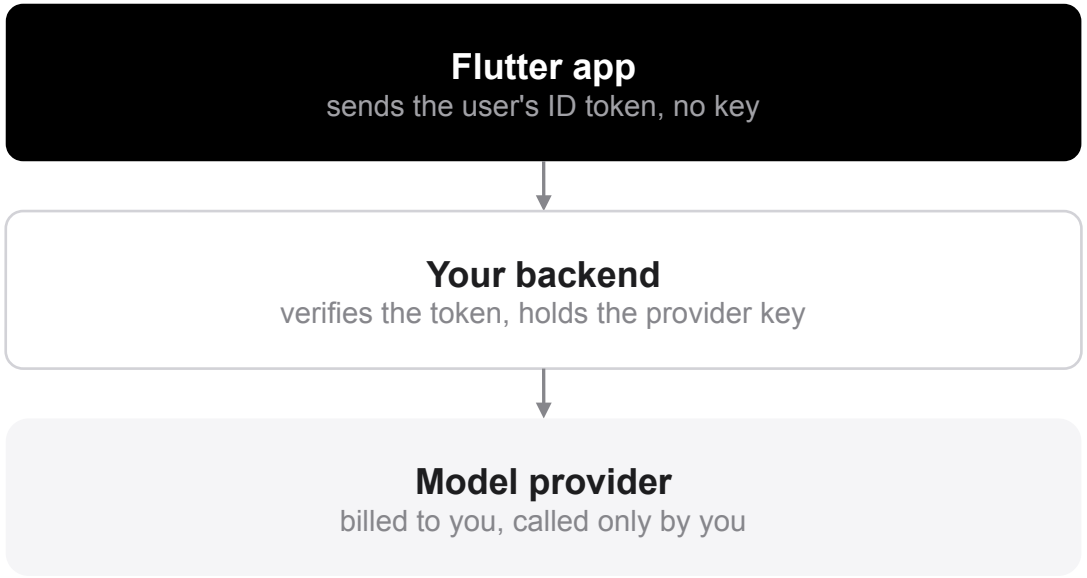
The bill is yours. A provider key is billed to the account that issued it, so a leaked key is someone else's inference at your expense until you happen to look.

Rotating the key ships a new app version and leaves every old install broken.

There is no secret you can ship inside a client. The same rule as Lecture 5, this time with a billing consequence.

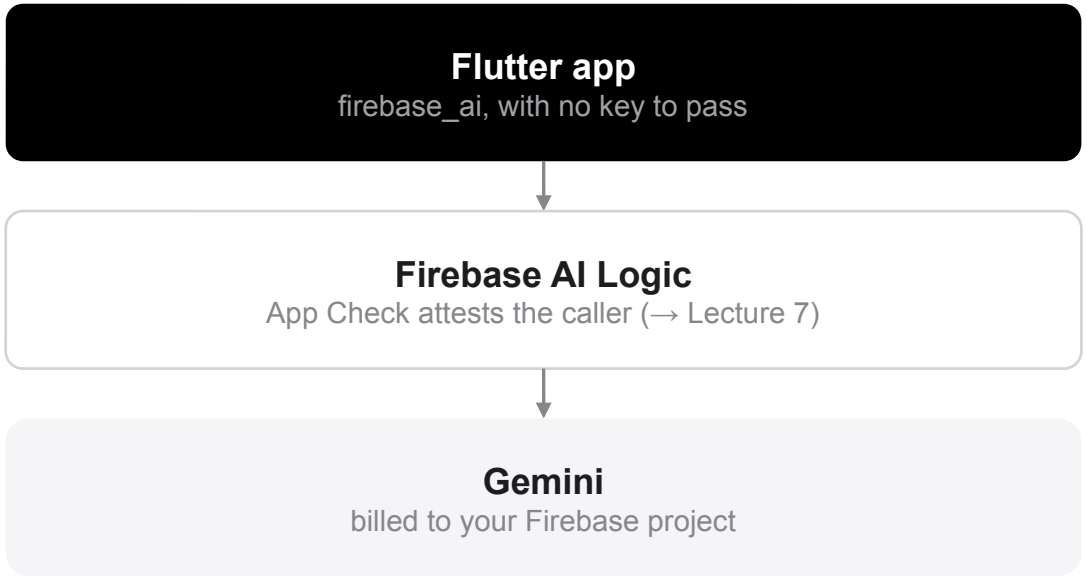
Two correct shapes, and what each one costs

A: your backend proxies the call



Any provider, your own rate limits, your own logging and prompt control, plus a service you have to operate (→ Lecture 5).

B: Firebase AI Logic + App Check



Working this afternoon, with attestation built in, but Google models only, and only if you press Enforce in App Check.

Both put the credential somewhere the user cannot reach. That is what separates them from the key-in-the-app version.

firebase_ai: the call with no key in it

```
// pubspec.yaml:  firebase_core, firebase_ai, firebase_app_check
await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform);

// App Check first: it attests this is a genuine build of YOUR app.
await FirebaseAppCheck.instance.activate(
  androidProvider: AndroidProvider.playIntegrity,
  appleProvider: AppleProvider.appAttest,
);

final model = FirebaseAI.googleAI().generativeModel(
  model: 'gemini-2.5-flash',
  systemInstruction: Content.system(
      'Answer only from the catalog you are given.'),
  generationConfig: GenerationConfig(
      temperature: 0.2, maxOutputTokens: 512),
);

final res = await model.generateContent([Content.text(question)]);
```

There is no apiKey parameter to pass. The credential is the project configuration, and the request is only served when it carries a valid App Check token (→ Lecture 7).

activate() only measures. Enforcement is a separate switch in the console, per product. That is the step that keeps scripts out.

maxOutputTokens is a cost control, not a style preference. It is the only hard cap you get.

temperature low, systemInstruction narrow: the cheapest steering available, and it runs before any of your validation does.

Stream the answer, do not freeze the screen

```
Stream<String> ask(String prompt) async* {
  final buf = StringBuffer();
  final s = _model.generateContentStream([Content.text(prompt)]);
  await for (final chunk in s) {
    buf.write(chunk.text ?? '');
    yield buf.toString();   // the UI rebuilds per chunk
  }
}

StreamBuilder<String>(
  stream: _answer,
  builder: (context, snap) => switch (snap.connectionState) {
    ConnectionState.waiting => const ThinkingLine(),
    _ => SelectableText(snap.data ?? ''),
  },
)
```

Time to first token is the latency users feel, not total time. Streaming turns a six-second wait into a 600 ms one without making anything faster.

Show three distinct states: sent, thinking, writing. A spinner that never changes is indistinguishable from a hang.

Give the user a cancel. Hold the StreamSubscription and cancel it on dispose, on navigation, and whenever a new question starts.

Nothing here can jank the UI unless you do work inside the builder. Keep the builder to layout.

A frozen screen is a bug, not a wait. A canceled request has still paid for the radio wake-up (→ Lecture 11).

PART 4 · ENGINEERING REALITY

Cost, failure, and answers that are wrong

The three areas to get right before you ship

ENGINEERING REALITY

Tokens: the unit you are billed in

~4

characters per token

for English prose; code and non-Latin scripts are worse

in + out

both are billed

and the whole prompt is re-sent on every single turn

quadratic

cost of a naive chat

resend the full history and n turns cost about $n^2/2$ messages

---

Cap the output with maxOutputTokens, and cap the input by summarizing or truncating the history instead of resending all of it.

Route by difficulty: a small, cheap model answers most questions, and only its failures reach the expensive one. That is the hybrid slide again, one layer up.

Identical prompts produce identical bills, so cache them, and cache the retrieval that built them.

Per-user quotas live on your backend. An app cannot enforce a quota against a user who controls the app (→ Lecture 5).

ENGINEERING REALITY

The failures you will actually see

	What it means	What to do about it
429	Rate limited, or over quota	Back off exponentially with jitter; cap the attempts
503 / 500	The provider is failing	Retry, but every retry is billed again
400	Your request is wrong	Never retry. Fix the prompt, the schema or the size
Timeout	No answer inside your budget	Cancel, say so, offer a retry. Do not hang silently
Safety block	The model declined to answer	A normal outcome, not an exception. Handle it as a state

Set a budget for the feature, not for the attempt: three tries of ten seconds each is a thirty-second stare at a spinner, which no user waits out.

Streaming changes the shape of a timeout: you can time out on time-to-first-token and let a long answer keep flowing.

Retries multiply load and cost, so they need a cap. Reuse the backoff-with-jitter machinery from Lecture 5.

ENGINEERING REALITY

Models produce plausible wrong answers

It is not a bug you can prompt away.

A language model returns a likely continuation, not a true one. It will invent a product code, a price, a citation or an API method in exactly the same confident register as a correct answer.

Constrain the output space.

A value the model must choose from a fixed set cannot be hallucinated into something new. JSON mode plus a schema turns free text into a parseable contract.

Then validate it anyway.

Parse it, range-check it, and look every id up in your own data before you believe any of it.

Show where the answer came from.

If it came from your data, show the row. An answer the user can check is worth far more than one they have to trust.

Treat model output like user input from the internet. The same trust boundary as Lecture 5, this time on the way back in.

```
generationConfig: GenerationConfig(
  responseMimeType: 'application/json',
  responseSchema: Schema.object(properties: {
    'category': Schema.enumString(
        enumValues: ['food', 'travel', 'other']),
    'amountCents': Schema.integer(),
  }),
),

// A schema is a constraint, not a guarantee.
final data = jsonDecode(res.text!);
if (!_allowed.contains(data['category'])) {
  return _askAHuman();
}
```

Never wire model output straight to consequence.
Money, a database write, a permission change, a message to another person, a device actuator: a rule or a human belongs in between.

Before next week

Recap

On-device or cloud is an architecture decision: latency, privacy, offline, cost, ceiling

ML Kit for solved problems, LiteRT for your own model, quantized so it fits

No app ever holds a provider key: your backend holds it, or Firebase AI + App Check does

Stream the answer, cap the tokens, and validate everything the model says

This week

Add ML Kit text or barcode scanning to your project app, and close the detector

Put one cloud call behind firebase_ai with App Check activated

Measure it: time to first token, tokens per call, cost per thousand users

Run strings over your own release build once and look for your key

Read more

firebase.google.com/docs/ai-logic

developers.google.com/ml-kit

ai.google.dev/edge/litert

pub.dev/packages/tflite_flutter

OWASP MASVS: Platform Interaction & Code Quality